Presentation and Demo Guide

Self-Supervised AI for ETL Error Detection and Root-Cause Localization

- Part 1: Running the demo
  - Before you walk in
  - The demo itself
  - If they ask to see more
  - If something goes wrong
  - Have these ready to open
- Part 2: Explaining the project in plain words
  - The thirty-second version
  - The two-minute version
  - Every piece, explained simply
  - Questions you will probably get
  - Three things to volunteer rather than defend
  - Two things to be careful with

Two parts. The first is how to run the live demo without anything going wrong. The second explains every piece of the project in plain words, so you can answer questions without falling back on jargon you would then have to defend.

Part 1: Running the demo

Before you walk in

Open a terminal, go to the project folder, and run this once to check everything is in place:

```
cd ~/Downloads/dl_project && ls data/etl_v1/dataset.npz runs/ssl/best.pt
```

If both files are listed, you are ready. If either is missing, the demo cannot run, and you need to rebuild it:

```
PYTHONPATH=src .venv/bin/python -m etlguard.data.build_dataset --n-batches
        8000 --out data/etl_v1
PYTHONPATH=src .venv/bin/python -m etlguard.train.pretrain --epochs 40 --
        out runs/ssl
```

That takes about 25 minutes, so do it the night before, not in the room.

Do a full practice run the day before. Not a glance at the command, an actual run, on the actual laptop, ideally on the projector. Most demo disasters are laptop problems rather than code problems.

The demo itself

One command, about ten seconds:

```
PYTHONPATH=src .venv/bin/python scripts/demo.py --n 2
```

It picks real broken pipeline runs from data the model has never seen, diagnoses each one, and then shows one healthy run so the contrast is visible.

What to say while it loads

> "This is taking a held-out pipeline run, one the model has never seen. It will answer three questions: is this run broken, which task broke it, and what kind of problem is it. Then it prints the real answer underneath so you can check it."

Reading the output

The output has four blocks. Walk through them in order.

Block 1, is it broken.

```
1. Is this run broken?   -> ALERT
   anomaly score 12.196   alert threshold 3.084
```

Say: "The score is how surprised the model is. The threshold is set so it would raise about one alert in twenty runs, which is a realistic amount of noise for an on-call engineer. This one is four times over."

Block 2, which task.

```
2. Which task caused it?
   1. extract_customers    ████████████████████ 1.000  <- most likely origin
   2. clean_customers      ████████████████████ 1.000
```

Say: "This is the hard part. Several tasks look bad, because the damage spreads. The bars are the model's ranking of which one actually started it."

Block 3, what kind of problem.

```
3. What kind of break?
   completeness   ████████████████████ 56.43
   schema         ████████████████     44.29

   Diagnosis: extract_customers shows an anomaly in the completeness family,
   driven most by null_rate_max.
```

Say: “It does not just say something is wrong. It says which kind of thing is wrong, and which measurement gave it away.”

Block 4, the answer key.

```
   ground truth: fault 'type_mismatch' injected at 'extract_customers'
   severity 0.01, 2 of 9 tasks affected
   task CORRECT   symptom family CORRECT
```

Say: “And here is what was actually injected, so you can see whether it was right. This one had severity 0.01, which is about as faint as they get, and it still found it.”

Then the healthy run. Point out that the score sits below the threshold and no alert fires. It matters that it does not cry wolf.

If they ask to see more

These all work and are worth having ready:

```
# a different set of incidents
PYTHONPATH=src .venv/bin/python scripts/demo.py --n 3 --seed 12

# a specific failure type
PYTHONPATH=src .venv/bin/python scripts/demo.py --fault unit_change

# the case it is bad at, if they push you
PYTHONPATH=src .venv/bin/python scripts/demo.py --fault row_loss_truncation
```

That last one is worth volunteering rather than waiting to be caught by. If it gets one wrong, say: “This is the failure type I know it handles badly, and I can explain exactly why”, then go to the explanation in Part 2.

If something goes wrong

It gets an answer wrong on stage. Good. Say “it is right about eight times in ten, and this is one of the two”, then explain the failure mode. Confidence about a known weakness reads far better than surprise.

The command errors. Do not debug in the room. Say “I have the recorded output here” and switch to a screenshot. Take screenshots of a successful run beforehand and keep them in a folder.

No laptop, or no projector. Have the figures printed or on a phone. Figure 2 from the report, the one with the coloured band, carries the main idea by itself.

Have these ready to open

- The report, at the results table
- `artifacts/dataset_report/fig2_propagation.png`, the blast radius picture
- `artifacts/results/fig7_attention.png`, the attention picture
- The GitHub repo, at the test folder, if anyone asks whether it is tested

---

Part 2: Explaining the project in plain words

The thirty-second version

> "Companies move data through pipelines: pull it out of one system, clean it, join it together, load it into a reporting database. These pipelines break in ways that do not crash anything, so nobody notices for weeks. I built a model that spots broken runs without being given any examples of failures, and, more importantly, points at which step in the pipeline actually caused it."

The two-minute version

> "The hard part is not spotting that something is wrong. It is working out where. When one step breaks, every step after it also looks broken, and often looks worse, because the damage builds up. So the loudest alarm is usually furthest from the real problem.
>
> My model handles this by asking a different question of each step. Not 'does this look bad' but 'does this look bad given what it was fed'. A step that produces bad output from bad input is a victim. A step that produces bad output from perfectly good input is where the trouble started.
>
> There was no dataset for this, because no public dataset labels which step caused a failure. So I built a simulator of a nine-step pipeline and injected fifteen kinds of realistic failure into the actual rows of data, which means the damage spreads on its own rather than because I scripted it."

Every piece, explained simply

What is ETL?

Extract, Transform, Load. It is the plumbing that moves data around a company. Pull data out of the systems that create it, clean and reshape it, and load it somewhere people can run reports on.

What is a pipeline task?

One step in that plumbing. “Pull today’s orders” is a task. “Join orders to customers” is a task. Mine has nine of them, and they run every hour.

What is a DAG?

Just the map of which task feeds which. Orders and customers have to be cleaned before they can be joined, and the join has to happen before the daily totals can be worked out. It has no loops, hence “acyclic”.

What is self-supervised learning?

Learning without being told the answers. The trick is to hide part of the input and make the model predict the hidden part, because the rest of the input is the answer key.

The everyday version: cover a word in a sentence and guess it. To be good at that you have to actually learn how language works. Same idea here. Cover part of a pipeline run and predict it, and to be good at that you have to learn how the pipeline normally behaves.

Why it matters here: nobody has a labelled list of past pipeline failures. They are rare, written up inconsistently, and the ones nobody noticed are still sitting in the history unmarked. A method that needs labels is a method nobody can use.

What is the model actually looking at?

Not the raw data. For every task, every hour, I compute 38 summary numbers, for example: how many rows came out, what fraction were empty, how many duplicate keys, how far behind the timestamps are, how long the task took. That set of 38 numbers is the “profile” of one task at one moment.

A whole pipeline run is nine profiles, one per task. The model looks at a window of the last 32 runs.

Why simulate the data instead of using real data?

Two reasons, and it is worth having both ready.

First, no public dataset exists that labels which step caused a failure. The datasets everyone uses are server and sensor readings, which have no pipeline structure at all, so the question cannot even be asked of them.

Second, company pipeline data is private and never released.

So building the benchmark is part of the contribution, not a shortcut around a missing one. This is the question you are most likely to be challenged on, so answer it confidently.

Why inject faults into the rows rather than into the numbers?

If I had corrupted the summary numbers directly, I would have had to decide by hand how the damage spreads downstream. The model would then be learning my rules about spreading, not anything about data pipelines. That would be circular.

By corrupting the actual rows, the next task genuinely reads broken data, and the damage spreads by itself.

What is a counterfactual label?

A fancy phrase for a simple check. For each broken run, I run the whole thing a second time with the fault removed and everything else identical, then compare. Any task whose numbers actually moved was genuinely affected. Any task that came out the same was not.

Why bother: the obvious approach is to say "if a task breaks, everything after it is broken too". That is wrong. An extra unused column changes nothing downstream. A slow task still produces correct data. Measuring gives 2.48 tasks affected per incident instead of the 3.94 that assumption claims.

What is a transformer, in one sentence?

A kind of neural network that looks at all the pieces of its input at once and learns which pieces to pay attention to, rather than reading them strictly in order.

Mine pays attention along two directions: back through time, so each task can see its own recent history, and across the pipeline, so each task can see the others.

What does "masking" mean?

Hiding part of the input on purpose during training. I hide two ways:

- Hide one kind of measurement at one task, for example all the "how empty is it" numbers.
- Hide one task's numbers entirely, so the model has to work out what that task should have produced from what its inputs looked like.

The second one is what makes the culprit-finding work.

Why hide a whole group instead of scattered numbers: because "average emptiness" and "worst emptiness" sit next to each other and are obviously related. The model could guess one from the other without ever looking at the rest of the pipeline. Hiding the whole group closes that shortcut.

How does it find the culprit?

At the moment being checked, it hides one task and everything downstream of that task, then asks the model to predict what that task should have produced.

- If the prediction is close, the task's output follows sensibly from its input. It is a victim.
- If the prediction is way off, the task produced something strange even though its inputs were fine. That is where the problem began.

Why hide the downstream tasks too: I did not, at first, and it was a real bug. The model could see that task's own downstream steps, which were also broken, and work backwards from them to the broken value. It was effectively peeking at the answer. Hiding them took accuracy from 63% to 77%.

What is calibration and why did it matter so much?

Some tasks are naturally harder to predict than others. If you just rank tasks by "who has the biggest error", the task that is always hardest to predict wins every time, whether or not anything is wrong with it.

The fix is to compare each task against its own normal behaviour rather than against other tasks. It is like grading on a curve per subject instead of comparing raw marks across different subjects.

This turned out to be the main finding. Two completely different ways of fixing the same problem, one statistical and one structural, gave identical results, 77.3% each, against 58.8% with no fix. That gap is bigger than the gap between any two of the seven methods I compared.

What do the numbers mean?

- **AUPRC**: how good the model is at separating broken runs from healthy ones. Higher is better, 1.0 is perfect. Better than plain accuracy here because only about 15% of runs are broken, so a model that says "healthy" every time would get 85% accuracy and be useless.
- **Hit@1**: how often the top-ranked task is genuinely the one that caused it. Mine is 79.7%.
- **Hit@3**: how often the real culprit is in the top three. Mine is 97.3%, which is the number that matters in practice, because an engineer will happily check three tasks.
- **MRR**: similar idea. If the right answer is ranked first you score 1, second gives you a half, third a third, and so on, averaged over all incidents.

What is an ablation?

Take out one piece of the design, retrain, and see if the results get worse. If they do not, that piece was not doing anything.

I did four, and none of them changed anything. That sounds like bad news but it is a real result, and each one has a reason rather than a shrug.

What is a confidence interval, and why does it matter here?

A range that says how sure you can be about a number. My model gets 79.7% and another method gets 81.8%, which looks like a loss. But the ranges overlap heavily, so the honest reading is that they are tied, not ranked.

Only 291 broken runs are in the test set, and with that few, differences of two or three points are noise.

Questions you will probably get

"Why doesn't your model beat the simple methods?"

> "It doesn't, and I say so in the report. The reason is that each task is only described by 38 numbers, and I have nearly 5000 clean examples, which is close to ideal for simple statistical methods. There wasn't much headroom. Five of the seven methods land within 0.03 of each other. What I did find is that how

> you score the tasks matters more than which model you use, and that's worth 18 points where the whole spread between methods is about 4."

"Isn't synthetic data a cop-out?"

> "It would be if a real dataset existed. It doesn't. No public dataset labels which pipeline step caused a failure, and company pipeline data isn't released. Building the benchmark was the only way to make the question measurable. I'm also clear in the report that the ranking of methods is more trustworthy than any single number."

"How do I know the simulator is realistic?"

> "The fault types come from the taxonomy in Google's and Amazon's published data validation systems, so they're real failure modes rather than invented ones. And I don't hand-write how damage spreads: I corrupt the actual rows and let the real transformations carry it downstream. That said, I chose how often and how severely faults happen, and I list that as a limitation."

"What's the weakest part?"

> "Two failure types, both involving row counts changing. It localizes those at 29% and 43%. I traced why: a join's output row count depends on how well the keys match up between its two inputs, and I don't measure that anywhere. So the model literally doesn't have the information it would need. The fix is an extra measurement, not a better model, and that's the first thing I'd do next."

"Did you find anything you didn't expect?"

> "Yes, and it's uncomfortable. Training longer made the model better at its training task and worse at finding the culprit, 80% down to 76%. The model works by being surprised by broken data, and if you train it long enough it gets good at predicting the broken data too, so the surprise disappears. That undermines how I chose when to stop training, and I've written it up as an open problem rather than hiding it."

"How do you know it isn't just memorising?"

> "The test data is the last quarter of the timeline, and the model only ever trained on the earlier part. It never sees the test period at all. I also split by time rather than randomly, because a random split would let it learn from next week to predict this week."

"Is it tested?"

> “59 automated tests. They check the simulator’s contracts, that healthy runs really are clean, that every fault type produces a measurable effect, and that the scoring code indexes the right task. Two of those tests exist because they caught real bugs.”

Three things to volunteer rather than defend

Saying these before you are asked is much stronger than conceding them after.

1. **The model doesn’t win.** Say it in the results slide. Explain why, and pivot to what you did find.
2. **I got something wrong and fixed it.** The peeking bug in the culprit-finder, and the claim I had to correct afterwards. This is the single best evidence that you did the work.
3. **Here’s the part it’s bad at.** Row-count faults, and exactly why.

Two things to be careful with

Don’t oversell it. It is a solid student project with an honest evaluation, not a production system. If asked whether a company could use it tomorrow, the answer is no, it needs validating against real telemetry first.

Check the citations before you go in. The literature review has 27 papers in a table, and a table is exactly where a marker’s eye lands on a wrong year. Look each one up on Google Scholar. It takes twenty minutes and it is the highest-risk detail in the whole submission.